Gasballoon Cockpit

A single-file HTML web app for gas balloon flight planning and in-flight monitoring: live position tracking, wind-based landing predictions, staged descent planning, weather stations, air traffic, and offline map caching - built for use on a tablet in the basket.

Current version: v260824.21-1720 (24.08.2026) - matches the `APP_VERSION` constant near the top of `index.html`'s script. Scheme: `vYYMMDD.zz-HHMM` (date, a same-day counter resetting to 01 each new day, build time) - shown in the app's bottom-left corner, useful for confirming a deployment picked up the latest build rather than a stale cached one. `cors_test.html`'s own version marker is kept in sync with this; `admin.html` is versioned independently.

Contents

What it is

One HTML file, no build step, no server of its own. Open it in a browser (or install it to a home screen as a PWA, with `apple-touch-icon.png`) and it runs entirely client-side, calling a handful of free public weather/mapping APIs directly from the browser, plus a small Cloudflare Worker for the two things a browser genuinely can't do on its own (see Architecture). A service worker (`sw.js`) caches map *tile* requests only - deliberately not weather/elevation data, which needs to stay fresh - so previously-viewed map areas remain available if the connection drops. It checks for its own updates every 30 minutes, not just on page reload, for genuine 24/7 unattended operation; a screen wake lock keeps the display on, with automatic re-acquisition if the browser ever releases it.

The hamburger menu's last entry, "About - more info!", opens this README as a PDF in a small viewer page (`readme_viewer.html`) with always-visible Print/Close buttons, rather than the browser's native PDF viewer directly.

Zoom-in/out buttons (and the very first automatic centring on GPS after boot) target the visually *visible* map area's centre, not the underlying map container's own geometric centre - relevant whenever the sidebar is open, since it overlays on top of the map rather than shrinking it (see the sidebar note under Architecture for why that distinction matters elsewhere too).

Architecture: login, flight profiles, and the Cloudflare Worker

The problem this solves

A browser page with no server of its own can't safely hold a real secret (a GitHub token, an API key meant to stay private) - anything embedded in `index.html`'s source is visible to anyone who views the page source. It also can't make certain API calls directly: some services block direct browser requests via CORS regardless of how the request is authenticated. Both problems are solved the same way here: a small **Cloudflare Worker** (`cloudflare-worker.js`) sits between the browser and the outside services that need either a real secret or a CORS-free path, holding the actual credentials server-side and exposing a narrow interface the app calls instead.

Login and the shared store

A 3-letter user code + password gate replaces what used to be a single fixed password. Credentials are validated against a shared user table stored in a GitHub Gist, reached through the Worker (never directly against `api.github.com` from the browser). A session stays valid 12 hours before the gate reappears.

The same Gist also stores named **flight profiles** - a full settings snapshot (every Settings field, emergency contact details, API keys) shared across anyone who knows a profile's 8-character name + 6-digit PIN. From the login screen, "Open existing" loads one immediately; "Create new" writes a brand-new one (starting with empty settings, since nothing has been configured yet this early) and makes it active. Up to 3 recently used profile *names* (never PINs) are remembered locally and offered as tap-to-fill chips at login and in Settings - the PIN always stays a manual re-entry, since remembering it would be the actual security-relevant part.

A profile can also be loaded **mid-session** from Settings → Flightprofile Data, even if the session started in local-only mode - useful if the decision to use a shared profile happens after Settings is already open. The currently active profile is shown both in the footer (next to the version number) and at the top of that same Settings section ("Currently loaded flightprofile: X", or "(local)" when none is active).

Closing Settings after changing anything prompts to save: to the currently-open profile (re-asks its PIN), to a brand-new named profile (asks for a name + PIN, rejecting an already-taken name), or only locally - the last option is genuinely always available, matching the pre-login-system `localStorage` behaviour unchanged. **Independent of all this**, Settings → Data & Backup also offers a plain **file export/import** (JSON, native share sheet on mobile) - a one-off transfer between two devices that doesn't touch the shared store at all.

The Cloudflare Worker

`cloudflare-worker.js` (deployed separately at Cloudflare, not itself one of this repo's static files) exposes two things, both gated by a shared `X-App-Secret` header the app sends on every call:

- **GET / PUT** - reads/writes the shared Gist (the login store described above). The real GitHub token is held only as a server-side Secret on the Worker, never sent to the browser.
- **`/aprs`** - a pure relay to `api.aprs.fi`, which blocks direct browser requests via CORS. The Worker forwards the request server-side (where CORS doesn't apply) and returns the result with the necessary headers added. This does **not** change how the aprs.fi

API key itself is managed - it stays a normal, per-installation Settings field, only relayed through the Worker rather than held by it.

`WORKER_URL`/`WORKER_APP_SECRET` (constants near the top of `index.html`'s script) are still technically visible client-side the same way a raw token would be, but their blast radius is much smaller: even leaked, they only grant read/write access to this one narrow store through this one Worker, not a real GitHub account's broader access (no creating arbitrary new gists, no access to other repos).

A live-tested, hard lesson from setting this up: a **fine-grained** GitHub token (`github_pat_...` prefix) cannot access the Gist API at all, regardless of what permissions are granted on it - confirmed via a live HTTP 403 ("Resource not accessible by personal access token"), since Gists sit outside fine-grained tokens' repository-scoped permission model entirely. A **classic** token (`ghp_...` prefix) with just the `gist` scope is what's actually needed. Separately, Cloudflare Workers' own `fetch()` does not add a `User-Agent` header automatically the way `curl` does - GitHub's API rejects any request without one, which produced a confusing HTTP 502 even with fully correct credentials until an explicit `User-Agent` was added to every outbound GitHub call.

AdminApp

`admin.html` (deployed alongside `index.html`, versioned independently) is a separate, master-password-protected page for managing the shared Gist directly: two editable tables (users: code+password; flight profiles: name+PIN+"has settings" indicator - all shown in plain text, since none of it is sensitive), a "Create new Gist" button for first-time setup, and its own Worker-URL/App-Secret fields. It keeps one raw-GitHub-token path of its own, clearly marked "setup only", purely for the one-time act of creating a brand-new Gist (the Worker itself can only read/write an *existing* one, not create one).

Why the sidebar matters for map-control positioning

The sidebar (`#sidebar`) is an absolutely-positioned *overlay* on top of the map, not a flexbox sibling that shrinks the map's own container - `#map` itself is always full-width underneath it. Every right-side map control (base-layer picker, zoom buttons, scale bar, wind-particle slider stack) is positioned relative to that full-width container via `positionSidebarHandle()`, which shifts them left by the sidebar's actual current width (zero when collapsed) - or, when the wind-sounding panel is open and extends further left than the sidebar (deliberately allowed on a narrow screen like an iPad, rather than being squeezed too narrow), by however far *that* panel reaches instead, whichever is greater. This is the one shared mechanism for all of it; nothing else should independently reposition these elements.

Main functions

Two main functions, switched via a pill toggle at the top of the map.

Landing Area (default)

Shows the balloon's live predicted trajectory: a cruise segment (green) transitioning into a descent (yellow) at a configurable time ("Initiate descent in"), continuously recalculated as position, altitude, and wind data update. A Monte Carlo scatter of likely landing points is shown as a shaded polygon around the predicted landing point, with a black cross marking its actual centre (computed from the raw scatter itself, not the containing polygon's own vertices, which would skew it toward the edge). The map view gently follows this polygon as it shifts

- only actually panning/zooming if it would otherwise fall outside the visible area, debounced so dragging a slider doesn't cause the map to jump on every intermediate value.

The descent point (a yellow cross on the trajectory) can be adjusted two ways that stay in sync: the "Initiate descent in" slider (10-minute steps, defaults to 20min), or dragging the cross itself along the trajectory.

Extended trajectory preview: click anywhere within roughly 60° of the trajectory's own direction, beyond where the normal prediction already reaches, to see it extended further out (blue) - a frozen snapshot from the moment of the click, useful for comparing the real flown track against an earlier prediction.

A one-time onboarding hint appears the first time this function is used.

Plan Descent

Two sub-functions, switched via a smaller toggle directly under the main "Plan Descent" button. Both show a one-time onboarding hint on first use. Switching back to Landing Area clears the descent path but leaves the reachable area/Monte-Carlo landing area/centre marker in place, with a "Clear staged descent area" button to remove them explicitly.

Quick Descent (default): click any point on the map to find the nearest reachable landing area and the descent path to it - an orange cross (the calculated descent point) and a yellow descent path, the same visual language as Landing Area's own live prediction. The reference trajectory truncates at the descent point once a plan is active, and extends as far as the "Initiate descent in" slider is set to. The descent point can be dragged to explore alternatives, or a new point clicked anywhere to restart.

Staged Descent: drag a royal-blue cross marker along a fixed 240-minute reference trajectory (independent of Quick Descent's own horizon slider) to choose when descent begins, then click within the resulting reachable area to search for a plan that pauses at one or more intermediate altitudes (chosen automatically where wind conditions genuinely differ) rather than descending continuously. Opens a side panel with a chart of the planned stages: altitude, wind at each level, duration, an estimated ballast requirement per transition, and markers for any wind shear, inversion, or isothermal layers encountered. Minimum stage separation and maximum intermediate stages live directly in this panel; its own two sliders (max descent time, inter-stage rate) sit side by side in one row.

Flight data box

A small draggable panel showing course (always 3 digits), speed (km/h and kn), climb/sink rate, and altitude (m AMSL, ft, flight level above a configurable transition altitude, and AGL). Refreshes on a fixed 0.5s cadence, decoupled from the underlying GPS fix rate. Snaps flush against whichever map edge it's dragged past; defaults to the left edge below the header. Dims along with the map in night mode.

Descent Parameters

"Initiate descent in" and "Descent rate" sit side by side; Descent rate's own track is a wedge shape that grows taller toward higher values, visualizing the accelerating descent speed. Below each: the computed distance/effective rate, plus (for Descent rate) the adiabatic lift bonus as a weight-equivalent ballast figure. Intercept AGL, Rate below, and Scatter sit in a second row, each with its own computed time-to-reach badge.

Trajectory physics notes

A few non-obvious modeling decisions worth knowing about if the predicted path ever looks surprising:

- **Cruise-phase vector hierarchy**: as long as a real GPS course/speed is currently known, that vector drives the entire cruise-phase extrapolation - the reference trajectory is the straight-line continuation of the presently observed course/speed, for as long as that observation is available. Only when there's no GPS signal (including temporarily mid-flight) does it fall back to the forecast wind model at the current altitude. Below 2kn (~3.7km/h) reported ground speed, GPS course is no longer trusted for this even when technically known, since course-over-ground at low speed becomes dominated by GPS position jitter rather than genuine displacement - falling back to forecast wind instead, consistent with the no-GPS case.
- **Hourly forecast wind resolution**: every simulation step resolves which hour's forecast to use via `floor(t/3600)`, not `round()` - hourly data represents the full hour starting at that mark, so 30 minutes into a flight is still within hour 0's forecast. Using `round()` previously switched a full 30 minutes too early whenever the next hour's forecast differed meaningfully, producing a visible snap in the shown path partway through.
- **Staged Descent's Monte-Carlo scatter** uses antithetic sampling (each random draw simulated together with its exact opposite), since wind uncertainty here is a constant bias accumulating over a whole multi-hour path rather than averaging out step-by-step - a small independent sample could otherwise land asymmetrically by chance and pull the centroid noticeably away from the deterministic best-found plan.
- **Staged Descent's reachable-area grid** is deliberately fine-resolution and shared identically between the drawn reachable area and the specific-point search, refined from the top 4 (not just the single best) grid candidates for a given target - a coarser grid or single-candidate refinement can converge on a mediocre local optimum well short of the actual clicked target.
- **GPS altitude vs. lat/lon**: many devices (especially desktop/laptop computers using WLAN/IP-based location instead of a real GPS chip) report a real latitude/longitude but never an altitude at all. The app holds the last known altitude value in that case; the "Alt AMSL" label reads "Alt AMSL (no GPS altitude - held)" in amber whenever no real altitude reading has ever been received, so this doesn't look like a stuck/broken reading.

Landing Area sidebar card

Shown for whichever landing-point marker is currently relevant (Landing Area's live prediction, Quick Descent's planned point, or Staged Descent's):

- **Ground wind** at the predicted landing time, with gusts (when the model supports them) as a second line.
- **Terrain near landing point**: elevation-based roughness (flat / moderate slope / steep), nearby tagged buildings or forest, the specific land cover at the exact point (meadow, farmland, forest, residential, water, etc.), and a red "CAUTION WATER!" override if the point falls inside a mapped lake/river/reservoir - checked via genuine polygon containment, not just proximity, on its own independent request queue so a failure elsewhere can't block it.
- **Sun at estimated landing**: azimuth (3 digits) and elevation, a direction arrow pointing at the sun's actual position, plus remaining time until evening civil twilight (the more relevant cutoff than sunset itself for visibility).
- **Moon phase**: a filled SVG silhouette (not an emoji, for consistent cross-device rendering) with illumination percentage, a

waxing/waning trend arrow, and rise/set times.

Wind sounding panel

A detailed vertical wind profile, opened via the button next to the E/H model badge in the header. Shows wind speed and direction from 0m AGL up to current altitude + 1750m, at the current position, the currently active planned descent point (Quick or Staged, whichever is active - updates live while dragging), or a marker dropped anywhere on the map while the panel is open. Deliberately shows the model's own raw pressure-level points connected by straight lines, not a smoothed curve implying more vertical resolution than actually exists.

The primary model gets full meteorological wind barbs (shaft pointing toward where the wind is coming FROM, full barb = 10kn, half barb = 5kn, pennant = 50kn) plus degree labels; comparison models (only those with actual coverage at the queried point are offered) overlay as thin colored curves with simple arrows pointing the opposite way (toward where the wind is blowing TO), which reads more intuitively for a quick multi-curve comparison. A comparison arrow's label turns red with a light red background whenever that model's direction deviates more than 25° from the primary's at that altitude. A "ø Avg" toggle overlays a genuine per-altitude weighted average (by grid resolution and model-run age) across whichever models are currently selected. Both the average toggle and the primary model itself are individually toggleable (thicker black border when active) - hiding the primary only affects what's drawn, its data still backs the deviation check regardless.

Y-axis is AMSL throughout, with intercept altitude marked by a dashed line and AGL shown only near the bottom (in a separate column from AMSL, to avoid overlap) since that's what matters most near the ground; a green line marks true 0m AGL separately from the intercept line. X-axis (wind speed) scales dynamically to the displayed data's actual range. A vertical dual-handle slider narrows the shown altitude band; using it for the first time reveals a matching horizontal slider below the chart for the wind-speed range.

The panel is anchored to the screen's own right edge at its full width (not squeezed by the sidebar's own position the way an earlier version was), deliberately allowed to extend into the map area on a narrow screen - see the Architecture section for how the map controls react to that.

While this panel is open, the map's main click handler (which otherwise drives Quick/Staged Descent's own click logic) does nothing at all, leaving clicks exclusively to the wind-sounding marker's own handler - without this, a tap meant only to place the sounding marker could also reach the descent-planning logic and silently trigger an unwanted plan recomputation.

Emergency messaging

Opened via the red warning-triangle button in the header. Builds a plain-text position report (aircraft registration, pilot name/mobile/second number, current position, altitude, course/speed, nearest town, landing-area centre + a Google Maps link if a plan is active, UTC time) and shows it as a two-part preview: a bold title line ("[reg] EMERGENCY POSITION REPORT") followed by the rest of the message - the title is genuinely bold in the UI (a small `<div>` rather than a single plain `<textarea>`, which can't do partial styling at all), while a hidden `<textarea>` still holds the complete message as the unchanged data source every send channel reads from.

Three ways to send: **WhatsApp** (one `wa.me` tab per configured SMS recipient - `wa.me` links only accept a single recipient each, unlike `sms:` URIs), **Email** (up to 5 configurable recipients, the original single field kept as “Recipient 1” so existing saved profiles keep working), and **Copy**. “Send on all channels” fires WhatsApp + SMS + Email together (SMS itself has no dedicated standalone button, by design, but remains reachable through this combined one). None of these are genuinely silent except email when EmailJS is configured - WhatsApp/SMS/plain email all open their respective app with the message pre-filled, requiring a manual “send” tap, since this page has no server of its own and genuinely cannot send anything silently in the background on its own.

Email delivery detail worth knowing: the `mailto:` fallback path supports multiple comma-separated recipients natively (a well-established part of the URI scheme itself). The EmailJS silent-send path does **not** rely on that same comma-trick - EmailJS’s own documentation describes its “To Email” template field as holding a single recipient address even when driven by a dynamic variable, with Cc/Bcc as the documented mechanism for more; whether a comma-list in one field would actually reach everyone isn’t something EmailJS documents as supported. Instead, `emailjs.send()` is called once per configured recipient in parallel, which works correctly regardless of how the underlying template’s To field happens to be set up.

Every attempt (successful or not) is logged into the Flight Log, exported alongside the regular track in GPX/KML.

Weather modeling

Wind is fetched per-altitude across 19 pressure levels (18 for the two Météo-France models, which don’t publish 975hPa) from the auto-selected model for the current location (ICON-D2 for DACH, ICON-EU for wider Europe, GFS globally), refreshed on a timer (default 15min). CAPE and wind gusts are fetched separately on a reduced schedule (every 3rd wind refresh) so an unsupported variable on a given model can’t take down the core wind fetch. ARPEGE Europe and AROME France (Météo-France) can be picked manually for comparison - MeteoSwiss’s ICON-CH1/CH2 and the UK Met Office’s models were deliberately not added, since neither publishes pressure-level wind data through Open-Meteo as of this writing.

Ensemble modeling: with a commercial Open-Meteo API key (Professional tier or higher - a base Standard commercial key is not enough, since Ensemble access specifically requires Professional/Enterprise), the Monte-Carlo scatter in all three functions draws on real ensemble members (ICON-D2-EPS / ICON-EU-EPS / GEFS, ~20-40 members) instead of an artificial age-based heuristic - each sample uses an actual member’s own wind profile. A badge next to the model name (“E” green / “H” gray/amber) shows which mode is active; amber specifically means “key present but this tier doesn’t include ensemble access”, distinct from plain gray (“no key at all”). Falls back to the heuristic automatically without a key or if the ensemble fetch fails. (Ensemble member keys use 2-digit zero-padded numbering - `_member01`, not `_member1` - detected directly from a live response rather than assumed, since assuming unpadded keys previously meant members 1-9 silently returned empty wind profiles.)

Commercial Open-Meteo key (separate from ensemble access, though the same key covers both if the tier includes it): when set, every Open-Meteo request switches to `customer-api.open-meteo.com`, which has no daily call limit, instead of the free tier’s 10,000/day cap - relevant for genuine 24/7 unattended operation, since a single wind-profile request already counts as several calls. A running “Open-Meteo calls today” counter is shown in the weather model popover regardless of which tier is active.

Map layers

Power lines: overhead line towers/transformers render as small, muted markers (neutral grey / muted blue-grey, distinct tones) rather than bright circles, so they don't visually compete with the lines themselves - spotting the wires is what matters, not each individual mast. Category checkboxes fully rebuild the layer instance from scratch on change, not just redraw/re-add it, since the underlying vector-tile library keeps its own internal tile cache tied to the instance regardless of map attachment. The combined tile source used here bundles several non-power layer types (telecoms, petroleum, water infrastructure) beyond just power; everything not explicitly styled as a power layer is hidden by default via a Proxy wrapper around the whole style definition, robust against not knowing every non-power layer name in advance. Telecom/antenna masts are shown separately (distinct magenta colour, bold outline, own Settings checkbox), since a physical mast is as real a hazard as a power tower even though the invisible-by-default rule would otherwise hide it too.

Airspace: real, class-filtered boundaries from openAIP's Core API, drawn on top of a combined raster tile base (still the only way to show airports/navaids/reporting points, since openAIP retired separate per-category tile endpoints in 2023). Settings checkboxes (Class A-G, Restricted, Prohibited, TMZ, RMZ) genuinely filter what's drawn, refetched as the map moves or the selection changes; an unrecognized type/class is shown regardless of checkbox state rather than silently hidden.

Terrain & Ground Features / "Unlandable Territory": the power-infrastructure checkboxes here mark genuine landing hazards (overhead lines, towers, substations), not just map clutter - a heading and short explanation above them makes that explicit.

APRS: looks up specific callsigns entered in Settings (not a category or area search - aprs.fi's API only supports looking up callsigns you already know, confirmed against their current official API docs and an independent third-party integration; the earlier "Balloons/Aircraft/Ships/…" category checkboxes never actually controlled the search and were removed). Routed through the Worker's `/aprs` endpoint (see Architecture) to work around CORS.

Lightning: only polls when rain is detected nearby, via Xweather.

Settings

Organized into color-coded groups: **Flightprofile Data** (load/create profiles mid-session, current profile status), **General** (units, cache radius, transition altitude), **Weather** (model selection, sonde data, METAR/station sources, lightning), **Air Traffic & Airspace**, **Terrain & Ground Features**, **Descent Planning** (adiabatic model), **Safety & Positioning** (external GPS, emergency contact - up to 5 email and 5 SMS recipients), **Data & Backup** (profile export/import, API keys & tokens).

Tokens and keys: every external service credential the app uses, gathered in one place and individually editable - Open-Meteo (commercial), openAIP, RapidAPI (air traffic), aprs.fi, Xweather, EmailJS. As with any client-side app, these are visible in the page source to anyone who views it; changing one asks for confirmation first.

Data sources

Source	Used for	Status
Open-Meteo	Wind profiles, elevation, CAPE, wind gusts, precipitation, ensemble members	Working. Optional commercial key removes the free tier's daily quota; Professional tier+ additionally unlocks ensemble access.
SondeHub	Radiosonde launches, for real sounding data instead of the model	Working
api.existenz.ch (SwissMetNet)	Swiss weather stations	Working
Bright Sky	German weather stations	Working
MeteoGate/E-SOH	Weather stations across the rest of Europe (33 countries)	Station list loads; per-station value parsing not fully confirmed against a live response
ADSBExchange (via RapidAPI) + OGN/glidernet.org	Air traffic, deduplicated where both sources report the same aircraft	Working
RainViewer	Rain radar	Working
Overpass API	Airspace, terrain/land-cover, roads, region names, airports (4 mirror fallbacks)	Working. Terrain check runs on its own independent queue, separate from other features. Region-limited mirrors excluded from the fallback list, since one could silently return "success, zero elements" outside its coverage rather than an honest, retriable failure.
Nominatim	Reverse geocoding for landing-area place names	Working
openAIP	Airspace tiles	Working
MetarCentral	Airport METARs	Working. Curated list of major European airports first, falling back to a live Overpass query elsewhere.
		Working, routed through the Cloudflare Worker's `/aprs`

aprs.fi	APRS station tracking	endpoint to work around the API's own CORS restriction (not fixable by calling it directly from the browser).
Xweather	Lightning (only polls when rain is detected nearby)	Working
Meteologix/Kachelmannwetter	Independent ground-conditions cross-check at the landing point, plus precipitation probability	Working (Business Starter tier). Surface-level only - a supplementary cross-check, not a wind-profile source. Also drawn as station markers on the weather-stations layer, refreshed every 20 minutes (not tied to map movement, unlike METAR).
GitHub Gist (via Cloudflare Worker)	Shared login store and flight profiles	Working

---

Running the regression tests

```
npm install jsdom --no-save
node test-suite.js
```

Requires Node.js and network access only to install `jsdom` - every test itself runs against mocked network calls, no real credentials or live services touched. Exits code 0 if everything passes, 1 otherwise; run this after any change to `index.html` before deploying, not just occasionally. `test-helpers.js` provides the shared jsdom harness (Leaflet/Chart.js stubs, silenced console noise, static checks for syntax/div-balance/TDZ-risk/missing-IDs) for writing further tests without rebuilding that setup again.

Currently 33 tests covering: static checks, a live app boot, the login flow (correct/incorrect credentials), new flight profile creation, the emergency WhatsApp channel messaging the right recipient (not the pilot's own number), the Settings save-prompt's profile-visibility logic, the APRS Worker-proxy routing, recently-used profiles, Settings panel positioning, a Staged Descent race condition, the Staged Descent chart's iPad height fix, the emergency message title/body split, multiple emergency email recipients (both the `mailto:` and EmailJS paths), and the profile file-import flow.

Several of these tests exist specifically because they caught a real, previously-shipped bug: a `<div>`-nesting error that blanked the entire page below the header, the Worker's missing `User-Agent` header, the WhatsApp button messaging the pilot's own number instead of any actual contact, a Staged Descent race condition where a second in-flight computation could resolve with stale data from an earlier marker position, and a profile-import button that opened a file picker but never actually processed the selected file.

Deployment

This is a static file with no server - after any change, `index.html` (and `admin.html`/`cloudflare-worker.js`/`cors_test.html` if touched) has to be re-uploaded to wherever it's hosted before the live site reflects it. **The Cloudflare Worker must be redeployed separately** whenever `cloudflare-worker.js` changes - updating `index.html` alone does not update the Worker's own running code.

On iOS specifically, both the page itself and the service worker can be cached persistently enough that a hard reload or removing/re-adding the home-screen icon may be needed to pick up a new version, despite the `Cache-Control: no-cache, no-store, must-revalidate` meta tags already set on the page itself.

Cloud file pickers (Dropbox, Google Drive, etc.) shown when uploading a file are controlled by iOS/the browser based on installed provider apps, not by this page.

Known limitations

- Adiabatic braking and Monte-Carlo scatter (without a commercial API key/real ensemble data) are approximations, not calibrated against real flight data - a planning aid, not a certified instrument.
- The landing-point terrain check is an approximation, not a true line-of-sight/obstacle-clearance calculation - ring-sampled elevation plus OSM tags, since actual obstacle heights aren't available from any free source used here.
- MeteoGate/E-SOH per-station value parsing is implemented but not yet confirmed against a live response.

License